BECS 22623
Implementation of Numerical Methods using Appropriate software

Solving Systems of Linear Equations

Department of Mathematics
Faculty of Science
University of Kelaniya

Outline

1. Backward Substitution
2. Gaussian Elimination
3. Forward Substitution
4. LU Factorization
5. Jacobi Method
6. Gauss-Seidel method

Solving a 3x3 upper triangular system using back-substitution : Background

Consider the upper triangular system

$$a_{11}x_1 + a_{12}x_2 + a_{13}x_3 = b_1$$

$$0x_1 + a_{22}x_2 + a_{23}x_3 = b_2$$

$$0x_1 + 0x_3 + a_{33}x_3 = b_3$$

Applying back-substitution, we can find the solution x_1, x_2, x_3 as follows:

$$x_3 = \frac{b_3}{a_{33}}$$

$$x_2 = \frac{b_2 - a_{23}x_3}{a_{22}}$$

$$x_1 = \frac{b_1 - a_{12}x_2 - a_{13}x_3}{a_{11}}$$

Solving a 3x3 upper triangular system using back-substitution: Background

Now consider the matrix form of the upper triangular system

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

and writing $x = (x_1, x_2, x_3)^T, b = (b_1, b_2, b_3)^T, A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ 0 & a_{22} & a_{23} \\ 0 & 0 & a_{33} \end{bmatrix}$,

the solution outlined above can be expressed as follows:

$$x(3) = \frac{b(3)}{A(3,3)}$$

$$x(2) = \frac{b(2) - A(2,3)x(3)}{A(2,2)}$$

$$x(1) = \frac{b(1) - A(1,2)x(2) - A(1,3)x(3)}{A(1,1)}.$$

Example

Consider the upper triangular system

$$
\begin{aligned}
3x_1 - 4x_2 + 5x_3 &= 2 \\
0x_1 + 3x_2 - 4x_3 &= -1 \\
0x_1 + 0x_2 + 5x_3 &= 5
\end{aligned}
$$

Create a matrix to store the coefficient matrix. Then create two vectors to store the right-hand vector, b, and your solution vector , x (you may pre-allocate this using ones or zeros).

Example

```
% Create your matrix and vector here
A1 =[3 -4 5;0 3 -4;0 0 5]
x = zeros(1,3)'
b =[2 -1 5]'

% Solve the first upper-triangular system here
x(3) = b(3)/A1(3,3)
x(2) = (b(2)-A1(2,3)*x(3))/A1(2,2)
x(1) =  (b(1)-(A1(1,2)*x(2))-(A1(1,3)*x(3)))/A1(1,1)
```

Exercise

Apply your program to find the solution of the system below (*Note: you only need to modify your coefficient matrix and right hand vector)*

$$x_1 - 2x_2 + x_3 = 2$$

$$0x_1 + 4x_2 - 3x_3 = 1$$

$$0x_1 + 0x_3 - 3x_3 = 3$$

Exercise

```
% Solve the second upper-triangular system here

A2 =[1 -2 1;0 4 -3;0 0 -3]
x1 = zeros(1,3)'
b1 =[2 1 3]'


x1(3) = b1(3)/A2(3,3)
x1(2) = (b1(2)-A2(2,3)*x1(3))/A2(2,2)
x1(1) =  (b1(1)-(A2(1,2)*x1(2))-(A2(1,3)*x1(3)))/A2(1,1)
```

Exercise

Now write a MATLAB function file (open a new m-file) that takes as input a 3x3 upper triangular matrix and a 3x1 right-hand vector and returns the solution to the corresponding upper triangular system. Apply your function to find the solutions of the upper triangular systems in previous examples

Exercise

```
function x = Back_substitition(A,b)
% A - 3*3 matrix
% x - 3*1 solution vector
% b - 3*1 column vector
x = zeros(1,3)';
% Solve the first upper-triangular system here
x(3) = b(3)/A(3,3);
x(2) = (b(2)-A(2,3)*x(3))/A(2,2);
x(1) =  (b(1)-(A(1,2)*x(2))-(A(1,3)*x(3)))/A(1,1);

end
```

```
% Apply your MATLAB function file here


 x3 = Back_substitition(A1,b)

 x4 = Back_substitition(A2,b1)
```

Formulating a general solution to an nxn upper triangular system using back-substitution

Observe the indexing of the vectors and matrix used in formulating the solution of the 3x3 upper triangular system

$$x(3) = \frac{b(3)}{A(3,3)}$$

$$x(2) = \frac{b(2) - A(2,3)x(3)}{A(2,2)}$$

$$x(1) = \frac{b(1) - A(1,2)x(2) - A(1,3)x(3)}{A(1,1)}.$$

We can generalize this for an nxn system by specifying the ith compoent of the solution vector $x(i)$ as follows

$$x(i) = \frac{1}{A(i,i)}\left\{ b(i) - \sum_{j=i+1}^{n} A(i,j)x(j) \right\}$$

Since we are applying back-substitution, we will begin by finding the last component of the solution $x(n)$ then proceed to $x(n-1), x(n-2) \ldots x(1)$. In this way the index i runs from $n, n-1, n-2, \ldots 1$.

Create a MATLAB function to solve an nxn upper triangular system using back-substitution.

```
function x = Generalize_BackSub(A,b)
n = length(b);
x = zeros(1,n)'; % Initialize the solution vector x as a zero vector
for i = n:-1:1
% Calculate the sum of off-diagonal entries.
     sum = 0;
     for j = i+1:n
         sum = sum + A(i,j)*x(j);
     end
     % Update the ith component of the solution vector x.
     x(i) = (b(i) - sum)/ A(i,i);
end
end
```

Example

Test your generalized MATLAB function for solving nxn upper-triangular systems by solving

$$3x_1 - 4x_2 + 5x_3 = 2$$

$$0x_1 + 3x_2 - 4x_3 = -1$$

$$0x_1 + 0x_3 + 5x_3 = 5$$

Compare the solution with your answer from the previous part!

Example

```
x = Generalize_BackSub(A1,b)
```

Apply your generalized MATLAB function for solving nxn upper-triangular systems to solve

$$x_1 + 2x_2 + x_3 - x_4 = 5$$

$$0x_1 - 4x_2 + x_3 + 7x_4 = 1$$

$$0x_1 + 0x_3 - 2x_3 + x_4 = 1$$

$$0x_1 + 0x_3 + 0x_3 - x_4 = 3$$

Example

```
A3 =[1 2 1 -1;0 -4 1 7; 0 0 -2 1;0 0 0 -1]
b3 = [5 1 1 3]'

x = Generalize_BackSub(A3,b3)
```

MATLAB *linsolve*

MATLAB has an inbuilt function ***linsolve(A,b)*** for solving linear equations To solve the linear system Ax = b you will enter the command

x = linsolve(A,b);

Where b is a column vector.

Now solve the upper triangular system. Do the results agree with the output from your code?

```
x = linsolve(A3,b3)
```

Implement Gaussian Elimination to solve a system of linear equations

In this Activity, you will learn how to write a MATLAB function to solve an nxn system of linear equations using $Ax = b$ ***Guassian Elimination****. Your function should first perform row reduction to create an upper triangular matrix and then back-substitution.*

Elimination step

The elimination step proceeds as follows. First you identify a column of interest and then you reduce all the entries below the main diagonal of that column to zero. We will accomplish this using two loops. The first loop will cycle through the columns of the matrix. Since we only need to reduce n-1 columns (no row reduction is needed in the last column of an upper triangular matrix) our counter will run from 1 to n-1. The psuedocode is as follows;

```
for j= 1:n-1
    reduce all enteries below the main diagonal of the coefficient matrix to zero
end
```

Elimination step

The second loop will be used to reduce all enteries below the main diagonal of the coefficient matrix to zero. Suppose we are in column j, then for each row (of that column) the following operations will be needed.

1) Identify a multiplier.

2) Multiply the pivot row by the multiplier and add it to the current row. Replace the current row with the new result.

3) Update the right-hand vector with the operation in step 2.

In column 1 there are n-1 row enteries that need to be set to zero, begining from the row 2 and running till row n. In column 2 there are n-2 entries that need to be set to zero, begining from row 3 and running to row n. So in column j there are n-j enteries that need to be set to zero, begining from row j+1 and running to row n.This can be implemented with the following code.

```
for i = j+1:n
    m = -A(i,j)/A(j,j)
    A(i,:) = m*A(j,:) + A(i,:)
    b(i) = m*b(j) + b(i)
end
```

Elimination step

we have the following code for the **elimination step**

```
for j= 1:n-1
    for i = j+1:n
        m = -A(i,j)/A(j,j)
        A(i,:) = m*A(j,:) + A(i,:)
        b(i) = m*b(j) + b(i)
    end
end
```

Back-substitution step

Complete the Gaussian Elimination function with your back substitution step from previous part.

Example

1) Use your Gaussian Elimination function to solve the following systems of 3x3 equations.

a)

$$2x_1 - 2x_2 - x_3 = -2$$

$$4x_1 + x_2 - 2x_3 = 1$$

$$-2x_1 + x_3 - x_3 = -3$$

Example

```
A = [2 -2 -1;4 1 -2;-2 1 -1]

b =[-2;1;-3]
n=3
x = Gaussian_Elimination(A,b,n)
```

Exercise

Use Gaussian Elimination method to solve the linear system of equations

b)

$$x_1 + 2x_2 - x_3 = 2$$

$$0x_1 + 3x_2 + x_3 = 4$$

$$2x_1 - x_2 + x_3 = 2$$

Exercise

```
A1 = [1 2 -1;0 3 1;2 -1 1]
b1= [2;4;2]
n1 =3

x1 = Gaussian_Elimination(A1,b1,n1)
```

MATLAB ml-divide

MATLAB has a **built-in function (ml-divide)** for solving systems of linear equations using Gaussian Elimination. To solve the system of equations $Ax = b$ you will need to run the command

```
x = A\b
```

Example

Apply ml-divide to solve the equations in #1, *a*, *b*. Compare your results with your implementation in #1.

```
% #1
%(a)
x2 = A\b
%(b)
x3 =A1\b1
```

Implementing Forward Substitution for 3x3 system of equations - Background

Consider the lower triangular system

$$a_{11}x_1 + 0x_2 + 0x_3 = b_1$$

$$a_{21}x_1 + a_{22}x_2 + 0x_3 = b_2$$

$$a_{31}x_1 + a_{32}x_3 + a_{33}x_3 = b_3$$

Applying **forward-substitution**, we can find the solution x_1, x_2, x_3 as follows:

$$x_1 = \frac{b_1}{a_{11}}$$

$$x_2 = \frac{b_2 - a_{21}x_1}{a_{22}}$$

$$x_3 = \frac{b_3 - a_{31}x_1 - a_{32}x_{32}}{a_{33}}$$

Implementing Forward Substitution for 3x3 system of equations - Background

Now consider the matrix form of the lower triangular system

$$\begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix}$$

and writing $x = (x_1, x_2, x_3)^T, b = (b_1, b_2, b_3)^T, A = \begin{bmatrix} a_{11} & 0 & 0 \\ a_{21} & a_{22} & 0 \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$, the solution outlined above can be expressed as follows:

$$x(1) = \frac{b(1)}{A(1,1)}$$

$$x(2) = \frac{b(2) - A(2,1)x(1)}{A(2,2)} \quad \text{----------(1)}$$

$$x(3) = \frac{b(3) - A(3,1)x(1) - A(3,2)x(2)}{A(3,3)}.$$

Example

Consider the upper triangular system

$$x_1 + 0x_2 + 0x_3 = -7$$

$$0.5x_1 + x_2 + 0x_3 = -2$$

$$-0.5x_1 + -\frac{7}{3}x_3 + x_3 = 6$$

1) Create a matrix to store the coefficient matrix. Then create two vectors to store the right hand vector, b and your solution vector , x (you may preallocate this using ones or zeros).

2) Use the expression for the solution (1) provided above to write a program to find the solution to the sytem. Your solution should be $\left(-7, \frac{3}{2}, 6\right)$

Example

```
% Create your matrix, right hand vector and solution vector
A = [1 0 0;0.5 1 0;-0.5 -7/3 1]
b = [-7;-2;6]
x = zeros(3,1)


% Solve the lower triangular system
x(1) = b(1)/A(1,1)
x(2) = (b(2)-A(2,1)*x(1))/A(2,2)
x(3) = (b(3)-A(3,1)*x(1)-A(3,2)*x(2))/A(3,3)
```

Exercise

Now write a MATLAB function file (open a new m-file) that takes as inputs a 3x3 lower triangular matrix and 3x1 right hand vector and returns the solution to the corresponding lower triangular system. Apply your function below to find the lower triangular system

$$x_1 + 0x_2 + 0x_3 = -2$$

$$2x_1 + x_2 + 0x_3 = 1$$

$$-x_1 + -\frac{1}{5}x_3 + x_3 = -3$$

Exercise

```
function x = Forword_substitition(A,b)
% A - 3*3 matrix
% x - 3*1 solution vector
% b - 3*1 column vector
x = zeros(3,1);
% Solve the first upper-triangular system here
x(1) = b(1)/A(1,1);
x(2) = (b(2)-A(2,1)*x(1))/A(2,2);
x(3) = (b(3)-A(3,1)*x(1)-A(3,2)*x(2))/A(3,3);

end
```

```
% Create your matrix, right hand vector and solution vector


A2 =[1 0 0;2 1 0;-1 -1/5 1]
b2 = [-2;1;-3]


% Run your MATLAB function to solve the linear system

x2 = Forword_substitition(A2,b2)
```

Implementing Forward Substitution for nxn system of equations

Observe the indexing of the vectors and matrix used in formulating the solution of the 3x3 lower triangular system

$$x(1) = \frac{b(1)}{A(1,1)}$$

$$x(2) = \frac{b(2) - A(2,1)x(1)}{A(2,2)} \quad \text{----------(2)}$$

$$x(3) = \frac{b(3) - A(3,1)x(1) - A(3,2)x(2)}{A(3,3)}.$$

We can generalize this for an nxn system by specifying the ith compoent of the solution vector $x(i)$ as follows

$$x(i) = \frac{1}{A(i,i)}\left\{ b(i) - \sum_{j=1}^{i-1} A(i,j)x(j) \right\} \text{----------(3)}$$

Since we are applying forward substitution, we will begin by finding the first component of the solution $x(1)$ then proceed to $x(2), x(3) \ldots x(n)$. In this way the index i runs froward from $1, 2, 3, \ldots n$.

Create a MATLAB function to solve an nxn lower triangular system using forward-substitution.

```
function x = Generalize_ForwordSub(A,b)
n = length(b);
x = zeros(1,n)'; % Initialize the solution vector x as a zero vector
for i = 1:n
% Calculate the sum of sub-diagonal entries.
    sum = 0;
    for j = 1:i-1
       sum = sum + A(i,j)*x(j);
    end
    % Update the ith component of the solution vector x.
    x(i) = (b(i) - sum)/ A(i,i);
end

end
```

Example

Test your generalized MATLAB function for solving nxn lower-triangular systems by solving

$$x_1 + 0x_2 + 0x_3 = -2$$

$$2x_1 + x_2 + 0x_3 = 1$$

$$-x_1 + -\frac{1}{5}x_3 + x_3 = -3$$

Compare the solution with your answer from the previous exercise.

Example

```
x2_new = Generalize_ForwordSub(A2,b2)
```

Solving Systems of Linear Equations using LU Factorization

In this activity, you will learn to solve a linear system by first finding the PA=LU factorization and then applying two steps of a forward and backsubstitution.

Consider the linear system

$$\begin{bmatrix} 4 & 2 & 0 \\ 4 & 0 & 2 \\ 2 & 5 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

1) Use MATLABs blackslash operator to find a solution

```
A3 = [4 2 0;4 0 2;2 5 3]
b3 =[2;4;6]

x3 = A3\b3
```

MATLAB *lu*

The matlab command [L,U,P]=lu(A) generates the matrices L,U, P of the factorization.

Solving Systems of Linear Equations using LU Factorization

2) Find the PA=LU factorization of the coefficient matrix.

```
[L,U,P]=lu(A3)
```

Solving Systems of Linear Equations using LU Factorization

3) Use your forward substitution code to find y in Ly = Pb, where b is the constant vector defined in the system above and P is your permutation matrix from the LU decomposition.

```
pb = P*b3
y =Generalize_ForwordSub(L,pb)
```

Solving Systems of Linear Equations using LU Factorization

4) Use your backsubstution code from previous part to solve for x in the upper triangular system Ux = y, where y is the solution from problem 3 and U is the upper triangular matrix from your LU Decomposition.

```
x4 = Generalize_BackSub(U,y)
```

Extracting the diagonal, lower triangular and upper triangular parts of a given matrix

Consider the matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

The diagonal (D), lower triangular(L) and upper triangular (U) parts of A are defined as follows

$$D = \begin{bmatrix} a_{11} & 0 & 0 \\ 0 & a_{22} & 0 \\ 0 & 0 & a_{33} \end{bmatrix}, L = \begin{bmatrix} 0 & 0 & 0 \\ a_{21} & 0 & 0 \\ a_{31} & a_{32} & 0 \end{bmatrix}, U = \begin{bmatrix} 0 & a_{12} & a_{13} \\ 0 & 0 & a_{23} \\ 0 & 0 & 0 \end{bmatrix}$$

Use MATLAB to extracting the diagonal, lower triangular and upper triangular parts of a given matrix

Matlab has functions for obtaining D, L and U from a given matrix A.

- To extract the diagonal part of A, run the command D= diag(A) **to get as matrix we need D= diag(diag(A))**
- To extract the lower triangular part of A, run the command L= tril(A) **to get with out main diagonal L= tril(A,-1)**
- To extract the upper triangular part of A, run the command **U**= triu(A) **to get with out main diagonal U= tril(A,1)**

Example

Find the diagonal, lower triangular and upper triangular parts of the matrix

$$A = \begin{bmatrix} 3 & 1 & -1 \\ 2 & -5 & 2 \\ 1 & 6 & 8 \end{bmatrix}$$

```
A1=[3 1 -1;2 -5 2;1 6 8]
% below 3 matrices should change in the discription

D1= diag(A1)
L1= tril(A1,-1)
U1= triu(A1,1)

Dnew = diag(D1)
```

Implement the Jacobi method

To solve the linear system $\mathrm{Ax} = b$, given some initial seed $x^{(0)}$, the algorithm for Jacobi method is

$$x^{(k)} = D^{-1}(b - (L + U)x^{(k-1)}), k = 1, 2, 3, \cdots$$

where D,L,U are the diagonal, lower triangular and upper triangular parts of the matrix A respectively. $x^{(k)}$ is the approximate solution at the kth iteration. The inverse of the diagonal part, D can be computed using the **inv** function in MATLAB.

Algorithm of the Jacobi method

INPUTS: A, the n by n coefficient matrix

b, the constant right hand vector

Itmax, the maximum number of iterations desired.

xold, the initial solution vector.

OUTPUTS: xnew, the approximate solution after itmax iterations.

REPEAT : $\begin{aligned} \text{xnew} &= D^{-1}(b - (L+U)\text{xold}) \\ \text{xold} &= \text{xnew} \end{aligned}$

UNTIL: The maximum number of iterations is reached.

MATLAB code for Jacobi method

```
function [xnew,vec] =Jacobi_method(A,b,Imax,xold)
n = length(b);
D= diag(diag(A)); % diaglonal matrix
L= tril(A,-1);    % lower triangular matrix with out main diagonal
U= triu(A,1);     % upper triangular matrix with out main diagonal

%Dnew = diag(D);
vec = zeros(n,Imax);
InvD =inv(D);
for i =1 :Imax
    xnew = InvD*(b-(L+U)*xold);
    %xnew = (b-(L+U)*xold)/(D);
    xold =xnew;
    vec(:,i)=xnew;
end

end
```

Example

Use 10, 20,30 iterations of your MATLAB function for the **Jacobi method** to find an approximate solution to the linear system

$$\begin{bmatrix} 3 & 1 & -1 \\ 2 & 4 & 1 \\ -1 & 2 & 5 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix}$$

starting with the solution $x = (0, 0, 0)$. Given an exact solution of $x = (2, -1, 1)$ Calculate the forward error in your approximation using the infinity norm.

Example

```
A2 = [3 1 -1 ;2 4 1;-1 2 5]
b2 =[4;1;1]
xold = [0;0;0]
Imax = 10
[xnew,vec] =Jacobi_method(A2,b2,Imax,xold)

xactual = [2;-1;1]
forword_error = max(abs(xactual-xnew))
```

Example

Modify your program to display the **forward error** after each iteration in a table.

```
count = 1:1:Imax;
for_error = zeros(1,Imax);

for i = 1 : Imax
    for_error(i) = max(abs(xactual-vec(:,i)));
end

disp('iteration #   Forward error')
fprintf('%8.4f      %8.4f\n',[count;for_error])
```

Implement the Gauss-Seidel method

To solve the linear system $\mathrm{Ax} = b$, given some initial seed $x^{(0)}$, the algorithm for Gauss-Seidel method is

$$x^{(k)} = (L + D)^{-1}(b - Ux^{(k-1)}), k = 1, 2, 3, \cdots$$

where D,L,U are the diagonal, lower triangular and upper triangular parts of the matrix A respectively. $x^{(k)}$ is the approximate solution at the kth iteration.

Algorithm of the Gauss-Seidel method

INPUTS: A, the n by n coefficient matrix

b, the constant right hand vector

Itmax, the maximum number of iterations desired.

xold, the initial solution vector.

OUTPUTS: xnew, the approximate solution after itmax iterations.

REPEAT : $\begin{aligned} \text{xnew} &= (L+D)^{-1}(b - U\,\text{xold}) \\ \text{xold} &= \text{xnew} \end{aligned}$

UNTIL: The maximum number of iterations is reached.

MATLAB code for Gauss-Seidel method

```
function [xnew,time_GS,vec] =Gauss_Seidel_method(A,b,Imax,xold)
n = length(b);
D= diag(diag(A)); % diaglonal matrix
L= tril(A,-1);    % lower triangular matrix with out main diagonal
U= triu(A,1);     % upper triangular matrix with out main diagonal

vec = zeros(n,Imax);
InvD = inv(L+D);
tic
for i =1 :Imax
    xnew = InvD*(b-(U)*xold);
    xold =xnew;
    vec(:,i)=xnew;
end
time_GS=toc;
end
```

Example

Use 10 iterations of your MATLAB function for the **Gauss-Seidel method** to find an approximate solution to the linear system

$$\begin{bmatrix} 3 & 1 & -1 \\ 2 & 4 & 1 \\ -1 & 2 & 5 \end{bmatrix} \begin{bmatrix} u \\ v \\ w \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \\ 1 \end{bmatrix}$$

starting with the solution $x = (0, 0, 0)$. Given an exact solution of $x = (2, -1, 1)$

- Calculate the forward error in your approximation using the infinity norm.
- How does your forward error compare with the error computed in privious part?

Example

```
[xnewGS,time,vecGS] =Gauss_Seidel_method(A2,b2,Imax,xold)
forwoed_error_GS = max(abs(xactual-xnewGS))
```

Department of Mathematics

Faculty of Science, University of Kelaniya.